CirculAR — Catálogo de enums y constantes oficiales

1. Objetivo

Definir el catálogo oficial inicial de enums, constantes operativas y valores controlados de CirculAR para evitar ambigüedad entre:

- producto
- dominio
- contratos API
- modelo relacional
- implementación backend
- frontend consumidor

Este documento fija una fuente única para:

- estados oficiales
- tipos operativos
- catálogos funcionales del MVP
- códigos de error
- razones de moderación
- quick actions
- límites operativos base

Se apoya en:

- CirculAR — Blueprint inicial
- CirculAR — Definiciones oficiales v2
- CirculAR — Dominio oficial v1
- CirculAR — Estados e invariantes v1
- CirculAR — Contratos API v1
- CirculAR — Arquitectura backend v1
- CirculAR — Modelo relacional inicial

---

2. Principios de catálogo

2.1 Fuente única de verdad

Los valores aquí definidos deben ser considerados oficiales para:

- enums de dominio

- enums persistidos en DB cuando corresponda
- DTO validation
- Swagger/OpenAPI
- frontend types
- renderizado de labels y badges

2.2 Separar código de label visible

Regla obligatoria:

- el backend y la DB operan con códigos estables
- el frontend decide cómo mostrar copy final al usuario
- nunca usar el label visible como valor persistido

Ejemplo correcto:

- código: `USED_GOOD`
- label UI: `Usado · Muy buen estado`

2.3 Catálogo controlado, no hipernormalizado

En MVP conviene mantener catálogos cerrados en código o configuración versionada, no como panel editable complejo.

2.4 Expansión compatible

Todo catálogo debe permitir:

- agregar nuevos valores sin romper integraciones
- deprecar valores con control
- mantener compatibilidad razonable entre backend y frontend

---

3. Estados oficiales del dominio

3.1 ListingState

```
export enum ListingState {
  DRAFT = 'DRAFT',
  IN_REVIEW = 'IN_REVIEW',
  OBSERVED = 'OBSERVED',
  PUBLISHED = 'PUBLISHED',
  PAUSED = 'PAUSED',
  RESERVED = 'RESERVED',
  CLOSED = 'CLOSED',
  REJECTED = 'REJECTED',
```

```
  ARCHIVED = 'ARCHIVED',
}
```

3.2 PurchaseIntentState

```
export enum PurchaseIntentState {
  ACTIVE = 'ACTIVE',
  ACCEPTED = 'ACCEPTED',
  REJECTED = 'REJECTED',
  CANCELLED = 'CANCELLED',
  EXPIRED = 'EXPIRED',
  CLOSED = 'CLOSED',
}
```

3.3 TradeProposalState

```
export enum TradeProposalState {
  ACTIVE = 'ACTIVE',
  ACCEPTED = 'ACCEPTED',
  REJECTED = 'REJECTED',
  CANCELLED = 'CANCELLED',
  EXPIRED = 'EXPIRED',
  CLOSED = 'CLOSED',
}
```

3.4 MatchSessionState

```
export enum MatchSessionState {
  OPEN = 'OPEN',
  ACTIVE = 'ACTIVE',
  COMPLETED = 'COMPLETED',
  FAILED = 'FAILED',
  EXPIRED = 'EXPIRED',
  CANCELLED = 'CANCELLED',
  CLOSED = 'CLOSED',
}
```

3.5 ConversationThreadState

```
export enum ConversationThreadState {
  OPEN = 'OPEN',
  RESTRICTED = 'RESTRICTED',
  CLOSED = 'CLOSED',
```

```
  ARCHIVED = 'ARCHIVED',
}
```

3.6 ModerationReviewState

```
export enum ModerationReviewState {
  PENDING = 'PENDING',
  APPROVED = 'APPROVED',
  OBSERVED = 'OBSERVED',
  REJECTED = 'REJECTED',
  SUPERSEDED = 'SUPERSEDED',
}
```

3.7 ImageAuditStatus

```
export enum ImageAuditStatus {
  PENDING = 'PENDING',
  APPROVED = 'APPROVED',
  OBSERVED = 'OBSERVED',
  REJECTED = 'REJECTED',
  ERROR = 'ERROR',
}
```

3.8 NotificationState

```
export enum NotificationState {
  UNREAD = 'UNREAD',
  READ = 'READ',
  ARCHIVED = 'ARCHIVED',
}
```

---

4. Tipos operativos base

4.1 InteractionType

```
export enum InteractionType {
  PURCHASE_INTENT = 'PURCHASE_INTENT',
  TRADE_PROPOSAL = 'TRADE_PROPOSAL',
}
```

4.2 MatchType

```
export enum MatchType {
  PURCHASE = 'PURCHASE',
  TRADE = 'TRADE',
}
```

4.3 MessageType

```
export enum MessageType {
  TEXT = 'TEXT',
  QUICK_ACTION = 'QUICK_ACTION',
  SYSTEM = 'SYSTEM',
}
```

4.4 ListingMode

Persistirlo es opcional. Si se persiste, este es el catálogo oficial:

```
export enum ListingMode {
  PURCHASE_ONLY = 'PURCHASE_ONLY',
  TRADE_ONLY = 'TRADE_ONLY',
  BOTH = 'BOTH',
}
```

Regla derivada:

- `allowsPurchase=true && allowsTrade=false` => `PURCHASE_ONLY`
- `allowsPurchase=false && allowsTrade=true` => `TRADE_ONLY`
- `allowsPurchase=true && allowsTrade=true` => `BOTH`

---

5. Catálogo comercial inicial del MVP

5.1 Categorías oficiales

MVP incluye ropa + calzado + accesorios.

```
export enum ListingCategory {
  TOPS = 'TOPS',
  BOTTOMS = 'BOTTOMS',
  OUTERWEAR = 'OUTERWEAR',
```

```
  DRESSES_AND_ONE_PIECE = 'DRESSES_AND_ONE_PIECE',
  FOOTWEAR = 'FOOTWEAR',
  BAGS = 'BAGS',
  ACCESSORIES = 'ACCESSORIES',
}
```

Labels sugeridos

- `TOPS` → Tops
- `BOTTOMS` → Bottoms
- `OUTERWEAR` → Abrigos
- `DRESSES_AND_ONE_PIECE` → Vestidos y enterizos
- `FOOTWEAR` → Calzado
- `BAGS` → Carteras y bolsos
- `ACCESSORIES` → Accesorios

5.2 Subcategorías iniciales

Catálogo inicial suficientemente útil para MVP, sin exceso de granularidad.

```
export const LISTING_SUBCATEGORIES: Record<ListingCategory, string[]> = {
  TOPS: [
    'TSHIRT',
    'SHIRT',
    'BLOUSE',
    'HOODIE',
    'SWEATER',
    'TOP',
    'POLO',
  ],
  BOTTOMS: [
    'JEANS',
    'PANTS',
    'SHORTS',
    'SKIRT',
    'LEGGINGS',
  ],
  OUTERWEAR: [
    'JACKET',
    'COAT',
    'BLAZER',
    'VEST',
  ],
  DRESSES_AND_ONE_PIECE: [
    'DRESS',
```

```
    'JUMPSUIT',
  ],
  FOOTWEAR: [
    'SNEAKERS',
    'BOOTS',
    'SANDALS',
    'HEELS',
    'FLATS',
    'LOAFERS',
  ],
  BAGS: [
    'HANDBAG',
    'BACKPACK',
    'TOTE',
    'SHOULDER_BAG',
    'WALLET',
  ],
  ACCESSORIES: [
    'BELT',
    'HAT',
    'SCARF',
    'JEWELRY',
    'SUNGLASSES',
    'WATCH',
  ],
};
```

Criterio

- mantener pocas categorías raíz
- dejar subcategorías suficientes para filtrar y moderar mejor
- no entrar todavía en hipersegmentación por estilo

5.3 Talles oficiales MVP

Como el MVP incluye ropa, calzado y accesorios, el catálogo de talles debe tolerar distintos formatos.

```
export enum GarmentSize {
  XXS = 'XXS',
  XS = 'XS',
  S = 'S',
  M = 'M',
  L = 'L',
  XL = 'XL',
  XXL = 'XXL',
```

```
  XXXL = 'XXXL',

  SIZE_34 = '34',
  SIZE_35 = '35',
  SIZE_36 = '36',
  SIZE_37 = '37',
  SIZE_38 = '38',
  SIZE_39 = '39',
  SIZE_40 = '40',
  SIZE_41 = '41',
  SIZE_42 = '42',
  SIZE_43 = '43',
  SIZE_44 = '44',
  SIZE_45 = '45',

  ONE_SIZE = 'ONE_SIZE',
}
```

Reglas

- ropa: `XXS` a `XXXL`
- calzado: `34` a `45`
- accesorios/carteras: `ONE_SIZE`

5.4 Condición de la prenda

```
export enum GarmentCondition {
  NEW_WITH_TAG = 'NEW_WITH_TAG',
  NEW_WITHOUT_TAG = 'NEW_WITHOUT_TAG',
  USED_EXCELLENT = 'USED_EXCELLENT',
  USED_GOOD = 'USED_GOOD',
  USED_FAIR = 'USED_FAIR',
}
```

Labels sugeridos

- `NEW_WITH_TAG` → Nuevo con etiqueta
- `NEW_WITHOUT_TAG` → Nuevo sin etiqueta
- `USED_EXCELLENT` → Usado · Excelente estado
- `USED_GOOD` → Usado · Muy buen estado
- `USED_FAIR` → Usado · Buen estado

5.5 Moneda oficial MVP

```
export enum CurrencyCode {
  ARS = 'ARS',
}
```

Decisión v1:

- MVP opera en Argentina
- solo `ARS` en precio publicado

---

6. Catálogo de quick actions

Quick actions guiadas para conversación post-match.

```
export enum QuickActionCode {
  PROPOSE_MEETING = 'PROPOSE_MEETING',
  SHARE_AVAILABILITY = 'SHARE_AVAILABILITY',
  CONFIRM_INTEREST_STILL_ACTIVE = 'CONFIRM_INTEREST_STILL_ACTIVE',
  MARK_ON_THE_WAY = 'MARK_ON_THE_WAY',
  REQUEST_CONTACT = 'REQUEST_CONTACT',
}
```

Labels UX sugeridos

- `PROPOSE_MEETING` → Proponer encuentro
- `SHARE_AVAILABILITY` → Compartir disponibilidad
- `CONFIRM_INTEREST_STILL_ACTIVE` → Confirmar si sigue activo
- `MARK_ON_THE_WAY` → Voy en camino
- `REQUEST_CONTACT` → Pedir contacto

Regla de producto

Las quick actions son ayudas guiadas, no reemplazan el estado del match ni crean transiciones implícitas por sí solas.

---

7. Catálogo de moderación

7.1 ModerationReasonCode

```
export enum ModerationReasonCode {
  PHOTO_BLURRY = 'PHOTO_BLURRY',
```

```
  PHOTO_TOO_DARK = 'PHOTO_TOO_DARK',
  PHOTO_TOO_FEW = 'PHOTO_TOO_FEW',
  PHOTO_DUPLICATED = 'PHOTO_DUPLICATED',
  PHOTO_CONTENT_NOT_ALLOWED = 'PHOTO_CONTENT_NOT_ALLOWED',
  NOT_A_GARMENT = 'NOT_A_GARMENT',
  CATEGORY_MISMATCH = 'CATEGORY_MISMATCH',
  MISSING_REQUIRED_DATA = 'MISSING_REQUIRED_DATA',
  LOW_LISTING_QUALITY = 'LOW_LISTING_QUALITY',
}
```

Clasificación sugerida

Corregibles

- `PHOTO_BLURRY`
- `PHOTO_TOO_DARK`
- `PHOTO_TOO_FEW`
- `PHOTO_DUPLICATED`
- `CATEGORY_MISMATCH`
- `MISSING_REQUIRED_DATA`
- `LOW_LISTING_QUALITY`

Críticas o potencialmente bloqueantes

- `PHOTO_CONTENT_NOT_ALLOWED`
- `NOT_A_GARMENT`

7.2 ReportReasonCode

```
export enum ReportReasonCode {
  INAPPROPRIATE_CONTENT = 'INAPPROPRIATE_CONTENT',
  FRAUD_SUSPICION = 'FRAUD_SUSPICION',
  HARASSMENT = 'HARASSMENT',
  SPAM = 'SPAM',
  OTHER = 'OTHER',
}
```

---

8. Catálogo de notificaciones

```
export enum NotificationType {
  PURCHASE_INTENT_RECEIVED = 'PURCHASE_INTENT_RECEIVED',
  TRADE_PROPOSAL_RECEIVED = 'TRADE_PROPOSAL_RECEIVED',
  INTERACTION_ACCEPTED = 'INTERACTION_ACCEPTED',
  INTERACTION_REJECTED = 'INTERACTION_REJECTED',
```

```
  NEW_CONVERSATION_MESSAGE = 'NEW_CONVERSATION_MESSAGE',
  LISTING_OBSERVED = 'LISTING_OBSERVED',
  LISTING_REJECTED = 'LISTING_REJECTED',
  RESERVATION_EXPIRING = 'RESERVATION_EXPIRING',
  RESERVATION_EXPIRED = 'RESERVATION_EXPIRED',
  MATCH_COMPLETED = 'MATCH_COMPLETED',
}
```

---

9. Catálogo de errores de negocio oficial

Este catálogo parte de Contratos API v1 y pasa a ser oficial para backend.

9.1 Auth / identity

```
export enum AuthErrorCode {
  INVALID_CREDENTIALS = 'INVALID_CREDENTIALS',
  EMAIL_ALREADY_IN_USE = 'EMAIL_ALREADY_IN_USE',
  PHONE_ALREADY_IN_USE = 'PHONE_ALREADY_IN_USE',
  UNAUTHORIZED = 'UNAUTHORIZED',
  FORBIDDEN = 'FORBIDDEN',
  SESSION_EXPIRED = 'SESSION_EXPIRED',
}
```

9.2 Listings

```
export enum ListingErrorCode {
  LISTING_NOT_FOUND = 'LISTING_NOT_FOUND',
  LISTING_NOT_EDITABLE = 'LISTING_NOT_EDITABLE',
  LISTING_NOT_AVAILABLE = 'LISTING_NOT_AVAILABLE',
  LISTING_NOT_PUBLISHED = 'LISTING_NOT_PUBLISHED',
  LISTING_ALREADY_RESERVED = 'LISTING_ALREADY_RESERVED',
  LISTING_ALREADY_CLOSED = 'LISTING_ALREADY_CLOSED',
  LISTING_CANNOT_BE_PAUSED = 'LISTING_CANNOT_BE_PAUSED',
  LISTING_CANNOT_BE_RESUMED = 'LISTING_CANNOT_BE_RESUMED',
  LISTING_CANNOT_BE_ARCHIVED = 'LISTING_CANNOT_BE_ARCHIVED',
  LISTING_RESERVATION_NOT_RENEWABLE =
'LISTING_RESERVATION_NOT_RENEWABLE',
}
```

9.3 Photos / moderation

```
export enum ModerationErrorCode {
  PHOTO_UPLOAD_INVALID = 'PHOTO_UPLOAD_INVALID',
```

```
  PHOTO_AUDIT_PENDING = 'PHOTO_AUDIT_PENDING',
  PHOTO_QUALITY_NOT_SUFFICIENT = 'PHOTO_QUALITY_NOT_SUFFICIENT',
  PHOTO_CONTENT_NOT_ALLOWED = 'PHOTO_CONTENT_NOT_ALLOWED',
  CATEGORY_IMAGE_MISMATCH = 'CATEGORY_IMAGE_MISMATCH',
  LISTING_REVIEW_NOT_APPROVED = 'LISTING_REVIEW_NOT_APPROVED',
}
```

9.4 Interactions

```
export enum InteractionErrorCode {
  SELF_INTERACTION_NOT_ALLOWED = 'SELF_INTERACTION_NOT_ALLOWED',
  PURCHASE_INTENT_ALREADY_EXISTS = 'PURCHASE_INTENT_ALREADY_EXISTS',
  TRADE_PROPOSAL_ALREADY_EXISTS = 'TRADE_PROPOSAL_ALREADY_EXISTS',
  TRADE_PROPOSAL_INVALID = 'TRADE_PROPOSAL_INVALID',
  PROPOSED_ITEM_NOT_OWNED = 'PROPOSED_ITEM_NOT_OWNED',
  PROPOSED_ITEM_NOT_AVAILABLE = 'PROPOSED_ITEM_NOT_AVAILABLE',
  PROPOSED_ITEM_ALREADY_COMMITTED =
'PROPOSED_ITEM_ALREADY_COMMITTED',
  INTERACTION_NOT_ACTIVE = 'INTERACTION_NOT_ACTIVE',
  INTERACTION_NOT_RESOLVABLE = 'INTERACTION_NOT_RESOLVABLE',
}
```

9.5 Match / conversation

```
export enum MatchErrorCode {
  MATCH_NOT_FOUND = 'MATCH_NOT_FOUND',
  MATCH_ALREADY_CLOSED = 'MATCH_ALREADY_CLOSED',
  MATCH_NOT_CONFIRMABLE = 'MATCH_NOT_CONFIRMABLE',
  CONVERSATION_NOT_FOUND = 'CONVERSATION_NOT_FOUND',
  CONVERSATION_CLOSED = 'CONVERSATION_CLOSED',
  QUICK_ACTION_NOT_ALLOWED = 'QUICK_ACTION_NOT_ALLOWED',
}
```

9.6 Generic

```
export enum GenericErrorCode {
  VALIDATION_ERROR = 'VALIDATION_ERROR',
  CONFLICT = 'CONFLICT',
  RATE_LIMITED = 'RATE_LIMITED',
  INTERNAL_ERROR = 'INTERNAL_ERROR',
}
```

---

10. Límites operativos oficiales v1

Estos límites deben existir como constantes centralizadas del backend.

10.1 Listings y fotos

```
export const LISTING_LIMITS = {
  MIN_PHOTOS_TO_SUBMIT: 2,
  MAX_PHOTOS_PER_LISTING: 8,
  MAX_DESCRIPTION_LENGTH: 1000,
  MAX_TRADE_PREFERENCE_NOTES_LENGTH: 280,
  MAX_ACTIVE_LISTINGS_PER_USER: 100,
  MAX_PRICE_ARS: 999999999,
} as const;
```

10.2 Archivos

```
export const FILE_LIMITS = {
  MAX_PHOTO_SIZE_BYTES: 8 * 1024 * 1024,
  MIN_PHOTO_WIDTH: 600,
  MIN_PHOTO_HEIGHT: 600,
  ALLOWED_IMAGE_MIME_TYPES: ['image/jpeg', 'image/png', 'image/webp'] as const,
} as const;
```

10.3 Interacciones

```
export const INTERACTION_LIMITS = {
  MAX_ACTIVE_PURCHASE_INTENTS_PER_USER: 50,
  MAX_ACTIVE_TRADE_PROPOSALS_PER_USER: 50,
  MAX_PROPOSED_ITEMS_PER_TRADE_PROPOSAL: 5,
  MIN_PROPOSED_ITEMS_PER_TRADE_PROPOSAL: 1,
} as const;
```

10.4 Match y conversación

```
export const MATCH_LIMITS = {
  RESERVATION_DURATION_HOURS: 24,
  CONVERSATION_ARCHIVE_DELAY_HOURS: 72,
  MAX_MESSAGE_LENGTH: 1000,
} as const;
```

10.5 Reach zones

```
export const PROFILE_LIMITS = {
  MAX_REACH_ZONES_PER_USER: 10,
  MAX_BIO_LENGTH: 280,
  MAX_FIRST_NAME_LENGTH: 80,
  MAX_LAST_NAME_LENGTH: 80,
  MAX_INSTAGRAM_HANDLE_LENGTH: 64,
} as const;
```

Notas

- `MIN_PHOTOS_TO_SUBMIT = 2` es una buena base para calidad mínima sin volver pesado el alta.
- `MAX_PHOTOS_PER_LISTING = 8` alcanza bien para MVP mobile-first.
- `MAX_PROPOSED_ITEMS_PER_TRADE_PROPOSAL = 5` permite permuta 1:N útil sin volver caótica la evaluación.

---

11. Reglas derivadas oficiales

Estas reglas no son solo copy de producto: deben existir como helpers o policies reutilizables.

11.1 Listing mode derivado

```
export function deriveListingMode(input: {
  allowsPurchase: boolean;
  allowsTrade: boolean;
}): ListingMode {
  if (input.allowsPurchase && input.allowsTrade) return ListingMode.BOTH;
  if (input.allowsPurchase) return ListingMode.PURCHASE_ONLY;
  if (input.allowsTrade) return ListingMode.TRADE_ONLY;
  throw new Error('Invalid commercial configuration');
}
```

11.2 Estados terminales

```
export const TERMINAL_LISTING_STATES = [
  ListingState.CLOSED,
  ListingState.REJECTED,
  ListingState.ARCHIVED,
] as const;

export const TERMINAL_PURCHASE_INTENT_STATES = [
  PurchaseIntentState.REJECTED,
```

```
  PurchaseIntentState.CANCELLED,
  PurchaseIntentState.EXPIRED,
  PurchaseIntentState.CLOSED,
] as const;

export const TERMINAL_TRADE_PROPOSAL_STATES = [
  TradeProposalState.REJECTED,
  TradeProposalState.CANCELLED,
  TradeProposalState.EXPIRED,
  TradeProposalState.CLOSED,
] as const;

export const TERMINAL_MATCH_SESSION_STATES = [
  MatchSessionState.COMPLETED,
  MatchSessionState.FAILED,
  MatchSessionState.EXPIRED,
  MatchSessionState.CANCELLED,
  MatchSessionState.CLOSED,
] as const;
```

11.3 Estados visibles en discovery

```
export const DISCOVERABLE_LISTING_STATES = [
  ListingState.PUBLISHED,
] as const;
```

11.4 Estados editables de listing

```
export const EDITABLE_LISTING_STATES = [
  ListingState.DRAFT,
  ListingState.OBSERVED,
] as const;
```

---

12. Catálogo de acciones disponibles de referencia

Esto no significa que todas deban modelarse como enum persistido, pero sí deben estar unificadas en types compartidos.

12.1 ListingAvailableAction

```
export type ListingAvailableAction =
  | 'canBuy'
  | 'canTrade'
```

```
  | 'canSave'
  | 'canUnsave'
  | 'canPause'
  | 'canResume'
  | 'canArchive'
  | 'canEdit'
  | 'canUploadPhotos'
  | 'canSubmitForReview'
  | 'canRenewReservation';
```

12.2 InteractionAvailableAction

```
export type InteractionAvailableAction =
  | 'canCancel'
  | 'canAccept'
  | 'canReject'
  | 'canViewInterestedProfile'
  | 'canViewProposedItems';
```

12.3 MatchAvailableAction

```
export type MatchAvailableAction =
  | 'canSendMessage'
  | 'canUseQuickAction'
  | 'canConfirmSuccess'
  | 'canMarkFailed'
  | 'canCancel'
  | 'canShareExternalContact';
```

---

13. Ubicaciones y alcance

Para MVP, y alineado con lo ya cerrado, no conviene hiperformalizar mapas complejos en el catálogo base.

Decisión oficial v1:

- `city` y `zone` serán strings controlados por validación de longitud y normalización
- no habrá todavía catálogo geográfico maestro obligatorio
- Rosario y CABA son los focos iniciales de lanzamiento, pero el modelo permite otras ciudades desde el inicio

---

14. Convenciones de nombres técnicas

14.1 Archivos TypeScript sugeridos

```
src/common/domain/enums/
  listing-state.enum.ts
  purchase-intent-state.enum.ts
  trade-proposal-state.enum.ts
  match-session-state.enum.ts
  conversation-thread-state.enum.ts
  moderation-review-state.enum.ts
  image-audit-status.enum.ts
  notification-state.enum.ts
  interaction-type.enum.ts
  match-type.enum.ts
  message-type.enum.ts
  listing-mode.enum.ts
  garment-condition.enum.ts
  garment-size.enum.ts
  listing-category.enum.ts
  quick-action-code.enum.ts
  moderation-reason-code.enum.ts
  report-reason-code.enum.ts
  notification-type.enum.ts

src/common/domain/constants/
  listing-limits.constants.ts
  file-limits.constants.ts
  interaction-limits.constants.ts
  match-limits.constants.ts
  profile-limits.constants.ts
```

14.2 Convención de exportación

- enums y límites deben salir desde un barrel file controlado
- evitar múltiples fuentes alternativas del mismo catálogo

---

15. Decisiones cerradas que este catálogo refuerza

15.1 Compra y permuta conviven como caminos nativos

Se refleja en:

- `ListingMode`
- `InteractionType`
- `MatchType`

15.2 1 listing = 1 prenda física

Se refleja en:

- unidad de publicación
- categorías y subcategorías pensadas por item único

15.3 Permuta estructurada 1 a N

Se refleja en:

- `MIN_PROPOSED_ITEMS_PER_TRADE_PROPOSAL = 1`
- `MAX_PROPOSED_ITEMS_PER_TRADE_PROPOSAL = 5`

15.4 Chat solo post-match

Se refleja en:

- `ConversationThreadState`
- `MessageType`
- `QuickActionCode`

15.5 Moderación como parte del núcleo operativo

Se refleja en:

- `ModerationReviewState`
- `ImageAuditStatus`
- `ModerationReasonCode`
- `ModerationErrorCode`

---

16. Qué no conviene hacer todavía

- convertir categorías y talles en tablas administrables desde panel
- crear taxonomías gigantes de moda desde el inicio
- soportar múltiples monedas
- soportar quick actions configurables por admin
- permitir labels UI mezclados con valores persistidos
- dispersar códigos de error en cada módulo sin catálogo central

---

17. Resultado esperado de esta etapa

Con este catálogo oficial, CirculAR queda listo para:

1. definir enums y constants reales del código backend
2. alinear DTO validation + Swagger + DB enums
3. tipar correctamente frontend y contracts
4. construir el scaffold técnico NestJS sin drift semántico
5. evitar strings mágicos en lógica de negocio

---

18. Próxima etapa recomendada

La siguiente pieza correcta es:

1. **Scaffold técnico NestJS v1**
2. **Roadmap de implementación por módulos con criterios de aceptación**
3. luego, si querés, **contratos internos de repositorios y application services base**